

Laboratorio 2

Introducción al Analizador Léxico FLEX

Enunciado:

El objetivo de esta práctica consiste en aprender a cómo utilizar el analizador léxico FLEX. Para este propósito se empezará describiendo brevemente cómo funciona un programa FLEX, cómo está estructurado y cómo compilarlo.

Breve introducción al analizador léxico FLEX.

¿Qué es un analizador léxico? Un analizador léxico es un programa que busca en, generalmente, grandes ficheros de texto, aquellas cadenas que se corresponden con unos patrones especificados en el analizador.

¿Qué es FLEX? FLEX es un analizador léxico bajo licencia GPL. Cada vez que se encuentre uno de los patrones especificados en FLEX se puede ejecutar un conjunto de acciones asociadas. FLEX es el analizador de dominio público compatible con el analizador léxico más frecuentemente utilizado: LEX (bajo sistema UNIX). FLEX (y LEX) genera, dada una especificación correcta de patrones y acciones, un programa en lenguaje C que puede ser compilado para obtener un programa ejecutable.

¿Cómo se especifican los patrones? Cada patrón es una *expresión regular*.

Algunos de los patrones que se pueden utilizar son (la lista completa está en el manual de FLEX):

Patrón	Significado	Patrón	Significado
a	Carácter 'a'	r*	Cero o más r
.	Cualquier carácter excepto \n	r+	Una o más r
[abc]	El carácter 'a', el 'b' o el 'c'	r?	Cero o una r
abc	La cadena 'abc'	r{1,3}	Entre 1 y 3 r

Patrón	Significado	Patrón	Significado
[cv-yA]	El carácter 'c', el 'A' o un carácter entre el 'v' y el 'y'	{nombre}	La expansión de la definición de "nombre"
[^aB-F]	Cualquier carácter excepto 'a' o un carácter entre 'B' y 'F'	\x	Lo que sea x, por ejemplo \n o \t o \a o \\ o \b o \c ...
" [a-wp]"	La cadena " [a-wp]"	r s	Una r o una s
^r	Una r al comienzo de línea	r/s	Una r sólo si le sigue una s

Más ejemplos de patrones en FLEX ¿Qué significan? ¿Alguna subcadena de la cadena indicada pertenece al patrón? Si la respuesta es sí ¿Cuáles son?

Patrón	Significado	Cadena	¿Cuáles?
<code>[abc]+</code>		Hfgccbbcbada	
<code>(abc)+</code>		hfgccbbcbaaab	
<code>abc[abc]*abc</code>		fabccbbccacabc	
<code>.+</code>		abc\naaa\n\n\n	
<code>0 [1-9][0-9]*</code>		00012ab000a90	
<code>0{1,3}1{2}0{,5}</code>		011 0 0100000	
<code>[]+</code>		asc a d d ff\n	
<code>[Bb][Ee][Gg][Ii][Nn]</code>		BeGin BbegiN	
<code>[^ \t\n.]+[.][^ \t\n.]+</code>		Hola.que tal.	

Variables importantes. Hay dos variables durante el proceso de análisis léxico cuyo contenido puede resultar muy interesante. Una es la variable `yytext` que almacena la cadena que ha coincidido con un patrón. La otra es la variable `yylen` que indica la longitud de la cadena a la que apunta `yytext`. El modo de funcionamiento por defecto de FLEX consiste en intentar reconocer siempre la cadena de mayor longitud que coincide con alguno de los patrones especificados.

¿Cuál es la estructura de un programa en FLEX? Consta de tres secciones.

En la **primera sección** (opcional) se especifican definiciones básicas que serán reutilizadas en otras secciones.

La **segunda sección** está compuesta de parejas: *patrón acción*. En la que *patrón* es una expresión regular y *acción* es un conjunto de sentencias en C de la forma `{sentencia1; sentencia2; ...; sentenciaN;}`. Estas sentencias especifican las acciones a realizar cuando se localiza el *patrón* asociado. Cada sección se separa de la siguiente mediante el empleo de una línea que contiene únicamente `%%`.

En la **tercera sección** (opcional) se especifican funciones auxiliares que, entre otras cosas, pueden ser utilizadas en la parte acción de la segunda sección.

Laboratorio Propuesto

```
DIGITO [0-9]
%%
{DIGITO}{DIGITO}*      { printf(" \n ;Un entero! %d",  atoi(yytext));}
[a-zA-Z_][a-zA-Z0-9]* { printf(" \n ;Una variable! %s",  yytext);
                        printf(" que tiene %d caracteres",  yyleng);
                        }
.
%%
;
```

Entrega 1. Ejercicio: ¿Qué hace este programa?

```
%{
    int num_lineas = 0, num_caracteres = 0;
}%
%%
\n      {++num_lineas; ++num_caracteres;}
.       {++num_caracteres;}
%%
int main() {
    yylex();
    printf("\n # de lineas = %d", num_lineas);
    printf("\n # de caracteres = %d\n", num_caracteres) ;
}
```

Entrega 2. Ejercicio: ¿Qué hace este programa?

Entrega 3. Diseñar patrones FLEX que reconozcan:

- a) Nombres de variables
- b) Números enteros con y sin signo
- c) Números reales en coma flotante con y sin signo.
- d) Números reales en notación exponencial con signo opcional y con punto también opcional
- e) Operadores matemáticos: =, +, -, *, /, DIV, MOD
- f) Paréntesis: (,)
- g) Comentarios: # esto es un comentario que acaba en una línea

Entrega 4. Investigar cómo se pueden incluir comentarios en un programa FLEX.

Entrega 5. Diseñar un programa que analice un texto de entrada y sustituya dos o más blancos seguidos por un único blanco y dos o más tabuladores por un único tabulador.

Entrega 6. Diseñar un programa FLEX que borre los comentarios que aparezcan en un fichero de texto (se suponen comentarios de una sólo línea que empiezan por el símbolo #, como el caso del patrón g de la Entrega 3).